

ddr_system_v4.0.yaml

DDR System v4.0 — Authoritative Specification expressed in certified YAML schema

This file IS the DDR System specification represented as its own DAG.

The nodes section encodes the canonical 9-tier topology with all DAG edges.

All normative sections from the specification document are represented here.

Conforms to: ddr_node_schema.yaml (JSON Schema 2020-12)

Source reference: DDR System Specification v4.0 (2026-02-26)

```
ddr_version: "4.0"

project: name: "DDR System v4.0 — Authoritative Specification" created: "2026-02-26" mode: full

active_tiers:
  • XPD
  • SIL
  • GPCL
  • FCL
  • CL
  • SAL
  • ICL
  • CDL
  • ISL
```

— §1 SYSTEM METADATA

system_metadata: status: Finalized date: "2026-02-26" scope: "Systems-, language-, and domain-agnostic" authority: "DDR Architecture Board" lineage: "Supersedes DDR v3.1.1 (Claude_v3)" single_source_of_truth: > This document is the exclusive normative specification for the DDR System. All prior versions are superseded. No conversation record, partial specification, or derivative document carries normative weight.

design_philosophy: - principle: "Minimize Design Complexity" description: > Every element earns its existence. No tier, edge type, operation, or rule exists without a concrete problem it solves. The system must be adoptable by a solo developer on day one and scale to enterprise without structural changes. - principle: "Avoid Premature Optimization" description: > The Core defines the minimum viable graph. Advanced analytical capabilities, inference engines, and domain-specific intelligence are delivered exclusively via optional Extensions. The Core never anticipates an Extension. - principle: "Maximize Structural Integrity" description: > The DAG is the single source of truth. Every node is traceable, every edge is typed, every mutation is validated. The system is correct by construction.

changes_from_prior: - area: "Tier count" prior: "11 tiers (8 mandatory + 3 optional)" current: "9 tiers (7 mandatory + 2 optional)" rationale: > ORL absorbed into GPCL; HIL and TDL unified into CL. Eliminates two tiers without information loss. - area: "Fork-Join" prior: "FCL forks to HIL||TDL, joins at SAL" current: "FCL optionally constrains a single CL; CL constrains SAL" rationale: "Eliminates fork complexity; single constraint merge point at SAL." - area: "Edge types" prior: "6 types" current: "4 types" rationale: > cites merged into derives (a citation IS a derivation relationship). reads and annotates unified into extends (both describe Extension-to-Core interaction with the same constraint: no Core mutation). - area: "Operations" prior: "11 operations" current: "7 operations" rationale: > RELOCATE removed (contradicted ID immutability). ABSTRACT/CONCRETIZE merged into INSERT with direction parameter. DETECT_ORPHAN and DETECT_CONTAMINATION subsumed by VERIFY. - area: "Node ID immutability" prior: "Contradicted by RELOCATE" current: "Absolute — no operation mutates a node ID" rationale: "Resolves the v3.1.1 RELOCATE contradiction definitively." - area: "ARE staging" prior: "Ambiguous DRAFT-in-Core" current: "Extension Candidate Pool — explicitly outside Core DAG" rationale: "Eliminates read-only model tension. ARE-inferred nodes cannot contaminate Core." - area: "Express Mode" prior: "5 groups" current: "4 groups; UNBUNDLE determinism rule added" rationale: "Realigned to new 9-tier structure. UNBUNDLE determinism resolves allocation ambiguity." - area: "Service Model" prior: "3-tier pricing" current: "Removed" rationale: "Premature optimization; commercial model is an operational concern, not a specification concern."

§2 FOUNDATIONAL AXIOMS

- axioms:
- id: AX-1 name: Traceability statement: > Every non-root node must cite at least one parent via a typed edge. implication: > Complete audit trails from intent to implementation; no orphaned requirements.
 - id: AX-2 name: Abstraction Ordering statement: > Technology and implementation specificity are deferred until logically necessary. implication: > Tiers above CL (XPD, SIL, GPCL, FCL) must contain no technology, hardware, or implementation references.
 - id: AX-3 name: Determinism statement: > Identical inputs produce unambiguous, mechanically verifiable outputs. implication: > Automated validation and compliance checking are possible.
 - id: AX-4 name: Universality statement: > The Core applies to all software systems regardless of domain, scale, or technology. implication: > No domain-specific assumptions in any Core tier.
 - id: AX-5 name: Extensibility statement: > Advanced analytical capabilities are delivered exclusively via optional Extensions. implication: > Core remains stable under Extension addition, modification, or removal.
 - id: AX-6 name: Declarative Integrity statement: > The Core is strictly declarative; all inference, optimization, and automated recommendation are Extension-only behaviors. implication: > Core structural invariants cannot be destabilized by analytical logic.
 - id: AX-7 name: DAG Acyclicity statement: > No citation chain may produce a cycle; causality flows in one direction only. implication: > Graph traversal is always terminable.

§3.1 NODE SCHEMA FIELDS

- node_schema_fields:
- property: id type: "TIER-N.M" description: "Immutable on assignment — no operation may change a node ID."
 - property: tier type: Enum description: "One of: XPD, SIL, GPCL, FCL, CL, SAL, ICL, CDL, ISL."
 - property: title type: String description: "Human-readable artifact label."

- property: content type: Text description: "Body constrained by the tier's atomic ruleset."
- property: parent_ids type: "List[String]" description: "≥1 for all non-root nodes; typed by edge."
- property: status type: Enum description: "DRAFT | ACTIVE | DIRTY | DEPRECATED | SUPERSEDED"
- property: version type: SemVer description: "Content version string."
- property: created type: "ISO 8601" description: "Creation timestamp."
- property: modified type: "ISO 8601" description: "Last modification timestamp."
- property: extension_annotations type: Map description: "Read-only Extension metadata; never modifies content."

§3.2 EDGE TYPE DEFINITIONS

edge_type_definitions: design_decision: > v3.1.1 defined 6 edge types. 'cites' merged into 'derives' (a citation for traceability IS a derivation relationship). 'reads' and 'annotates' unified into 'extends' (both describe Extension-to-Core interaction with the same structural constraint: no Core mutation). Reduces vocabulary from 6 to 4 without losing expressiveness. types: - type: derives symbol: "—derives—▶" semantics: > Child content derived from parent requirements or references parent for traceability. - type: constrains symbol: "----constrains--▶" semantics: > Parent sets enforceable limits on child's design space. - type: implements symbol: "—implements—▶" semantics: > Child provides concrete realization of parent's abstract specification. - type: extends symbol: "...extends...▶" semantics: > Extension adds metadata to or reads Core node without modifying it.

§3.5 DAG INVARIANTS

dag_invariants:

- id: INV-1 statement: "No cycles permitted at any path length."
- id: INV-2 statement: > No tier-skipping: each citation references exactly one active tier above in the derivation path. Exception: FCL→SAL derives edge is always valid regardless of CL activation state (SAL merge-node design; see §3.4, Audit C-2).
- id: INV-3 statement: > XPD and CL are conditionally activatable; the Core is valid and complete without them.
- id: INV-4 statement: "When CL is inactive, SAL derives directly from FCL."
- id: INV-5 statement: "All non-root nodes must carry at least one parent_id citation."
- id: INV-6 statement: > At most one XPD node may carry status ACTIVE at any time. SUPERSEDE of an XPD node must atomically set the predecessor to SUPERSEDED before the replacement node can be set to ACTIVE.

§3.6 NODE ID FORMAT

node_id_format: general_pattern: "[TIER]-[SECTION].[ITEM]" xpd_pattern: "XPD-0.N (no sections; section = 0)" examples: - "SIL-1.3" - "GPCL-2.1" - "CDL-12.5" - "XPD-0.1" immutability_rule: > IDs are immutable once assigned. A superseded node retains its original ID with status SUPERSEDED; the replacement receives a new ID. No operation — including relocation — may alter a node's assigned ID.

§3.7 CITATION RULES

citation_rules:

- rule_id: CIT-R1 statement: "Every non-root node must have ≥ 1 parent_id."
- rule_id: CIT-R2 statement: > parent_ids must reference node(s) from the immediately preceding active tier(s) in the DAG topology.
- rule_id: CIT-R3 statement: > CL→SAL constraint edges are recorded in parent_ids with edge type 'constrains'.
- rule_id: CIT-R4 statement: > An inline [TIER-N.M] citation in node content must have a matching entry in parent_ids.
- rule_id: CIT-R5 statement: > Extension extends edges are stored in extension_annotations only — never in parent_ids.

§4 CONSUMPTION MODES

consumption_modes:

- mode: "Express (4 Groups)" description: "Adjacent tiers bundled into groups; expandable via UNBUNDLE." best_fit: "Small-to-medium projects."
- mode: "Full (9 Tiers)" description: "Every tier independently specified." best_fit: "Complex, regulated, or enterprise systems."

express_mode: description: > Express Mode is not a reduced system — it is Full Mode with grouped presentation. The UNBUNDLE operation expands any group into its constituent tiers without information loss or invention. When unbundling, parent_ids automatically wire to the immediately superior unbundled tier, satisfying CIT-R2 without manual intervention. groups: - group_id: G1 tiers: [XPD, SIL, GPCL] label: "Purpose, Strategy & Governance" - group_id: G2 tiers: [FCL, CL] label: "Capabilities & Constraints" - group_id: G3 tiers: [SAL, ICL] label: "Architecture & Contracts" - group_id: G4 tiers: [CDL, ISL] label: "Design & Scaffolding"

unbundle_determinism_rule: > Within Express Mode groups containing conditionally activatable tiers (G1: XPD+SIL+GPCL; G2: FCL+CL), content must be authored with explicit tier annotations (e.g. [FCL] or [CL] inline prefixes) to enable deterministic UNBUNDLE allocation. The UNBUNDLE operation must reject content that cannot be unambiguously assigned to a constituent tier.

§5 TIER DEFINITIONS

tier_definitions:

Tier 0: XPD

- tier_id: XPD label: "Existential Purpose Document" layer_label: "OPTIONAL ROOT" core_question: > What human or societal need does this system exist to address, and what ethical boundaries govern all downstream decisions? is_optional: true is_terminal_leaf: false is_merge_node: false activation_condition: > ethical_impact $\neq$ none OR societal_scale > personal. Required for AI/ML, healthcare, civic, and public-facing systems. Skippable for internal tooling with no external effect. root_when: "Always root when active. No parent." parent_relationships:

- tier: NONE edge_type: derives condition: none_root child_relationships:
- tier: SIL edge_type: derives condition: always atomic_inclusion_rules:
- rule_id: XPD-R1 statement: "Must articulate a fundamental human or societal need being addressed." violation_consequence: "Downstream tiers lack ethical grounding."
- rule_id: XPD-R2 statement: > Must be immutable across the project lifecycle; changes require a new XPD version. violation_consequence: "Scope drift; mission confusion."
- rule_id: XPD-R3 statement: > Must be comprehensible to non-technical stakeholders without a glossary. violation_consequence: "Stakeholder misalignment."
- rule_id: XPD-R4 statement: > Must establish ethical boundary conditions all subsequent tiers must satisfy. violation_consequence: "Unethical design without detection."
- rule_id: XPD-R5 statement: "Must define success criteria independent of implementation metrics." violation_consequence: "Wrong success measurement."
- rule_id: XPD-R6 statement: > Must identify populations who could be harmed and the safeguards required. violation_consequence: "Harm by omission." atomic_exclusion_rules:
- rule_id: XPD-E1 statement: > Must not contain solution concepts, technology references, or architectural ideas.
- rule_id: XPD-E2 statement: "Must not contain quantitative performance targets (→ GPCL)."
- rule_id: XPD-E3 statement: "Must not contain regulatory or legal constraints (→ GPCL)."

— Tier 1: SIL

- tier_id: SIL label: "Strategic Intent Layer" layer_label: "INTENT LAYER" core_question: > Why does this system exist, and what business outcomes must it achieve? is_optional: false is_terminal_leaf: false is_merge_node: false root_when: "Root when XPD is inactive." parent_relationships:
 - tier: XPD edge_type: derives condition: if_xpd_active
 - tier: NONE edge_type: derives condition: if_xpd_inactive_then_root child_relationships:
 - tier: GPCL edge_type: derives condition: always atomic_inclusion_rules:
 - rule_id: SIL-R1 statement: "Must define the core business problem or opportunity being addressed." violation_consequence: "GPCL will lack strategic anchor."
 - rule_id: SIL-R2 statement: "Must specify strategic objectives with measurable outcomes." violation_consequence: "Unmeasurable success criteria."
 - rule_id: SIL-R3 statement: > Must identify all stakeholder categories and their value propositions. violation_consequence: "Misaligned delivery priorities."
 - rule_id: SIL-R4 statement: > Must establish explicit scope boundaries (in-scope and out-of-scope). violation_consequence: "Uncontrolled scope creep."
 - rule_id: SIL-R5 statement: "Must define organizational success metrics." violation_consequence: "Inability to declare completion."
 - rule_id: SIL-R6 statement: "Must be stable under technology changes." violation_consequence: "Technology coupling at the intent level." atomic_exclusion_rules:
 - rule_id: SIL-E1 statement: > Must not reference hardware, technology stacks, frameworks, or languages.
 - rule_id: SIL-E2 statement: > Must not contain regulatory mandates or compliance requirements (→ GPCL).
 - rule_id: SIL-E3 statement: > Must not prescribe architectural patterns or implementation strategies.
 - rule_id: SIL-E4 statement: "Must not contain quantitative performance metrics (→ GPCL)."

— Tier 2: GPCL

- tier_id: GPCL label: "Governance, Policy & Quality Layer" layer_label: "GOVERNANCE LAYER" core_question: > What non-negotiable external mandates, regulatory obligations, policy constraints, and measurable quality thresholds govern this system? is_optional: false is_terminal_leaf: false is_merge_node: false design_decision: > ORL Absorption: v3.1.1 separated governance constraints (GPCL) from operational requirements (ORL) as independent tiers. In practice, operational quality thresholds (latency, availability, security) are themselves governance constraints — non-negotiable acceptance criteria imposed by external or organizational authority. Merging eliminates a tier boundary that created pass-through nodes without independent semantic value, while preserving classification distinction via content sections within GPCL. parent_relationships:
 - tier: SIL edge_type: derives condition: always child_relationships:
 - tier: FCL edge_type: derives condition: always atomic_inclusion_rules:

- rule_id: GPCL-R1 statement: > Must enumerate all applicable regulatory frameworks with jurisdiction and scope. violation_consequence: "Compliance gaps leading to legal exposure."
- rule_id: GPCL-R2 statement: > Must specify enforceable, testable constraints — not aspirational targets. violation_consequence: "Non-verifiable compliance claims."
- rule_id: GPCL-R3 statement: > Must identify contractual obligations imposed by third-party relationships. violation_consequence: "Contract breach by design."
- rule_id: GPCL-R4 statement: "Must define data sovereignty and residency requirements." violation_consequence: "Data law violations."
- rule_id: GPCL-R5 statement: "Must specify audit and record-retention mandates." violation_consequence: "Regulatory audit failure."
- rule_id: GPCL-R6 statement: > Must specify quantifiable performance targets: latency, throughput, concurrency ceilings. violation_consequence: "Architecture unable to satisfy operational demands."
- rule_id: GPCL-R7 statement: > Must specify reliability and availability targets (SLAs, RTO, RPO). violation_consequence: "Unacceptable service degradation."
- rule_id: GPCL-R8 statement: > Must specify security requirements expressed technology-neutrally. violation_consequence: "Stale security specification on technology change."
- rule_id: GPCL-R9 statement: "Must specify scalability and accessibility requirements." violation_consequence: "Architecture unable to grow; user exclusion."
- rule_id: GPCL-R10 statement: "Must cite parent SIL IDs for each constraint." violation_consequence: "Orphaned requirements." atomic_exclusion_rules:
- rule_id: GPCL-E1 statement: > Must not specify technology frameworks, library choices, or hardware specifications.
- rule_id: GPCL-E2 statement: "Must not describe functional system behaviors (→ FCL)."
- rule_id: GPCL-E3 statement: "Must not contain business objectives or success metrics (→ SIL)."

— Tier 3: FCL

- tier_id: FCL label: "Functional Capability Layer" layer_label: "FUNCTIONAL LAYER" core_question: > What externally observable behaviors and user-facing capabilities must the system provide? is_optional: false is_terminal_leaf: false is_merge_node: false parent_relationships:
 - tier: GPCL edge_type: derives condition: always child_relationships:
 - tier: SAL edge_type: derives condition: always
 - tier: CL edge_type: derives condition: if_cl_active atomic_inclusion_rules:
 - rule_id: FCL-R1 statement: > Must describe capabilities from the perspective of a user or external system. violation_consequence: > Internal implementation details contaminate the functional spec.
 - rule_id: FCL-R2 statement: > Must specify user workflows end-to-end without naming components, classes, or modules. violation_consequence: "Premature structural coupling."
 - rule_id: FCL-R3 statement: > Must define event-driven behaviors and conditional business logic rules. violation_consequence: "Missing behavioral specification."
 - rule_id: FCL-R4 statement: > Must specify user-observable state transitions and error conditions. violation_consequence: "Incomplete behavioral model."
 - rule_id: FCL-R5 statement: > Must be decomposable into sub-capabilities (parent-child FCL nodes) for complex features. violation_consequence: "Monolithic feature specs that resist traceability."
 - rule_id: FCL-R6 statement: > Must cite parent GPCL IDs for capabilities that satisfy a governance or quality requirement. violation_consequence: "Disconnected functional requirements." atomic_exclusion_rules:
 - rule_id: FCL-E1 statement: "Must not name specific classes, modules, APIs, or algorithms."
 - rule_id: FCL-E2 statement: > Must not specify network protocols, serialization formats, or data schemas.
 - rule_id: FCL-E3 statement: "Must not specify hardware requirements or infrastructure topology."

— Tier 4: CL

- tier_id: CL label: "Constraint Layer" layer_label: "CONSTRAINT LAYER" core_question: > What are the declared technology selections, hardware envelopes, and infrastructure ceilings that bound the system's implementation? is_optional: true is_terminal_leaf: false is_merge_node: false activation_condition: > Specific technology, hardware, or infrastructure constraints are non-negotiable. Optional when full freedom is preserved into the architecture phase. design_decision: > HIL/TDL Unification: v3.1.1 modeled hardware constraints (HIL) and technology constraints (TDL) as two independent, parallel tiers forming a fork-join topology. The unified CL eliminates fork-join complexity, the CRR protocol, and the parallel-tier invariant enforcement, while preserving full constraint expressiveness via content sections within a single tier. Conflicts between hardware and technology constraints are resolved internally within CL nodes. parent_relationships:

- tier: FCL edge_type: derives condition: if_cl_active child_relationships:
- tier: SAL edge_type: constrains condition: if_cl_active atomic_inclusion_rules:
- rule_id: CL-R1 statement: > Must declare approved programming languages with version constraints. violation_consequence: "Incompatible implementations."
- rule_id: CL-R2 statement: > Must declare mandatory frameworks and core libraries with minimum version bounds. violation_consequence: "Dependency drift."
- rule_id: CL-R3 statement: > Must declare required external service contracts without their internal implementation details. violation_consequence: "Integration gaps."
- rule_id: CL-R4 statement: > Must declare runtime environment constraints (OS, container runtime, execution environment). violation_consequence: "Deployment environment incompatibility."
- rule_id: CL-R5 statement: > Must explicitly declare prohibited technologies with rationale. violation_consequence: "License compliance violations."
- rule_id: CL-R6 statement: > Must declare hardware envelopes when applicable (CPU class, RAM floor, storage, GPU). violation_consequence: "Architecture that exceeds target hardware."
- rule_id: CL-R7 statement: > Must declare infrastructure ceilings when applicable (compute budget, storage cap, bandwidth cap). violation_consequence: "Cost overruns from unconstrained architecture."
- rule_id: CL-R8 statement: > Must specify deployment topology declarations (on-premise, cloud-agnostic, hybrid, edge). violation_consequence: "Architecture incompatible with deployment target."
- rule_id: CL-R9 statement: "Must cite FCL IDs for each constraint." violation_consequence: "Constraints untraceable to a business need."
- rule_id: CL-R10 statement: > Must explicitly document internal reconciliations of conflicting hardware and technology constraints. violation_consequence: > Loss of deterministic traceability for constraint conflicts.
- atomic_exclusion_rules:
- rule_id: CL-E1 statement: > Must not auto-derive, infer, or recommend configurations (→ Extensions).
- rule_id: CL-E2 statement: "Must not contain functional system behaviors (→ FCL)."
- rule_id: CL-E3 statement: "Must not contain cost models or TCO calculations (→ Extensions)."

— Tier 5: SAL

- tier_id: SAL label: "System Architecture Layer" layer_label: "ARCHITECTURE LAYER" core_question: > How is the system structurally decomposed, and what patterns govern component interaction? is_optional: false is_terminal_leaf: false is_merge_node: true parent_relationships:
 - tier: FCL edge_type: derives condition: always
 - tier: CL edge_type: constrains condition: if_cl_active child_relationships:
 - tier: ICL edge_type: derives condition: always atomic_inclusion_rules:
 - rule_id: SAL-R1 statement: > Must define the overarching architectural pattern(s) with rationale. violation_consequence: "No structural framework for downstream design."
 - rule_id: SAL-R2 statement: > Must specify system decomposition into major subsystems with ownership boundaries. violation_consequence: "Ambiguous component responsibilities."
 - rule_id: SAL-R3 statement: "Must specify inter-subsystem communication patterns." violation_consequence: "Integration design without architectural mandate."
 - rule_id: SAL-R4 statement: "Must specify concurrency model and data ownership rules." violation_consequence: "Race conditions and data integrity violations by design."
 - rule_id: SAL-R5 statement: "Must specify failure isolation and resilience boundaries." violation_consequence: "Cascading failure scenarios in the architecture."
 - rule_id: SAL-R6 statement: > Must cite all active parent IDs (FCL + CL if active) for each major architectural decision. violation_consequence: "Architectural decisions without traceable justification."
 - atomic_exclusion_rules:
 - rule_id: SAL-E1 statement: > Must not contain exact data schemas or payload definitions (→ ICL).
 - rule_id: SAL-E2 statement: "Must not contain class-level component blueprints (→ CDL)."
 - rule_id: SAL-E3 statement: > Must not contain executable code, algorithm implementations, or procedural logic (→ CDL/ISL).

— Tier 6: ICL

- tier_id: ICL label: "Interface & Contracts Layer" layer_label: "CONTRACT LAYER" core_question: > What are the formal, machine-verifiable contracts governing data exchange between system boundaries? is_optional: false is_terminal_leaf: false is_merge_node: false parent_relationships:
 - tier: SAL edge_type: derives condition: always child_relationships:
 - tier: CDL edge_type: implements condition: always atomic_inclusion_rules:
 - rule_id: ICL-R1 statement: > Must define all inter-component and external API contracts with complete input/output schemas. violation_consequence: "Implementations that diverge at integration points."
 - rule_id: ICL-R2 statement: > All schemas must be machine-parseable (JSON Schema, Protobuf, OpenAPI, or equivalent). violation_consequence: "Contracts that cannot be mechanically validated."
 - rule_id: ICL-R3 statement: > Must specify serialization formats, encoding standards, and wire protocols per contract. violation_consequence: "Interoperability failures from encoding mismatches."
 - rule_id: ICL-R4 statement: > Must specify mandatory fields, optional fields, type constraints, and validation rules. violation_consequence: "Runtime failures from malformed payloads."
 - rule_id: ICL-R5 statement: > Must specify error response contracts (error codes, payload structure, retry behavior). violation_consequence: "Undefined failure behavior at system boundaries."
 - rule_id: ICL-R6 statement: "Must specify versioning strategy per contract." violation_consequence: "Breaking changes without migration path."
 - rule_id: ICL-R7 statement: "Must cite SAL IDs for each contract." violation_consequence: "Contracts without architectural justification." atomic_exclusion_rules:
 - rule_id: ICL-E1 statement: > Must not contain internal component state management or business logic.
 - rule_id: ICL-E2 statement: "Must not specify architectural routing patterns (→ SAL)."
 - rule_id: ICL-E3 statement: "Must not contain class or module blueprints (→ CDL)."

— Tier 7: CDL

- tier_id: CDL label: "Component Design Layer" layer_label: "DESIGN LAYER" core_question: > What are the structural blueprints of individual components — their public interfaces, internal state, and responsibilities? is_optional: false is_terminal_leaf: false is_merge_node: false parent_relationships:
 - tier: ICL edge_type: implements condition: always child_relationships:
 - tier: ISL edge_type: implements condition: always atomic_inclusion_rules:
 - rule_id: CDL-R1 statement: > Must define component names, logical responsibilities, and ownership boundaries. violation_consequence: "Ambiguous implementation targets."
 - rule_id: CDL-R2 statement: > Must specify all public method/function signatures (name, parameter types, return type, exceptions). violation_consequence: "Implementations that violate the declared interface."
 - rule_id: CDL-R3 statement: > Must specify internal state structures as a logical model — not implementation. violation_consequence: "Hidden state dependencies between components."
 - rule_id: CDL-R4 statement: > Must specify component dependencies (consumed components and ICL contracts). violation_consequence: "Circular dependencies introduced at implementation."
 - rule_id: CDL-R5 statement: "Must map each component to the ICL contracts it implements." violation_consequence: "Components without contractual grounding."
 - rule_id: CDL-R6 statement: > Must specify initialization, lifecycle, and teardown contracts for stateful components. violation_consequence: "Resource leaks and initialization-order bugs."
 - rule_id: CDL-R7 statement: > When CL declares multiple target languages, must produce language-specific blueprints for each target. violation_consequence: "Language constraint not propagated; ISL-R5 compliance gap." atomic_exclusion_rules:
 - rule_id: CDL-E1 statement: > Must not contain executable code bodies or algorithm implementations.
 - rule_id: CDL-E2 statement: "Must not contain system-wide architectural patterns (→ SAL)."
 - rule_id: CDL-E3 statement: "Must not contain data serialization schemas (→ ICL)."

— Tier 8: ISL

- tier_id: ISL label: "Implementation Scaffold Layer" layer_label: "SCAFFOLD LAYER" core_question: > What is the minimal, structurally valid, traceable scaffolding required to initiate implementation? is_optional: false is_terminal_leaf: true is_merge_node: false parent_relationships:
 - tier: CDL edge_type: implements condition: always child_relationships: [] atomic_inclusion_rules:

- rule_id: ISL-R1 statement: > Must produce syntactically valid structural scaffolding in the target language. violation_consequence: "Scaffolding that fails to compile or parse."
- rule_id: ISL-R2 statement: > Must embed docstrings or code comments with explicit parent DDR node IDs. violation_consequence: "Implementations without traceability."
- rule_id: ISL-R3 statement: "Must include implementation hints as structured comments." violation_consequence: "Implementers who lose architectural context."
- rule_id: ISL-R4 statement: > Must define all function/method bodies exclusively as stubs. violation_consequence: "Pre-implementation contamination."
- rule_id: ISL-R5 statement: > Must be language-specific — one ISL node per target language/runtime when multiple are declared in CL. violation_consequence: "Language-ambiguous stubs."
- rule_id: ISL-R6 statement: "Must cite CDL parent IDs for every stub." violation_consequence: "Orphaned scaffolding." atomic_exclusion_rules:
- rule_id: ISL-E1 statement: "Must not contain business logic or complete algorithmic logic."
- rule_id: ISL-E2 statement: "Must not contain infrastructure configuration (→ Extensions)."

§6 CONSTRAINT PRECEDENCE

constraint_precedence: tiers: - priority: 1 tier: XPD rationale: "Ethical boundary conditions are inviolable." - priority: 2 tier: SIL rationale: "Strategic intent defines the purpose of all design decisions." - priority: 3 tier: GPCL rationale: "External regulatory mandates and quality thresholds are non-negotiable." - priority: 4 tier: FCL rationale: "Functional requirements operate within the constraint envelope." - priority: 5 tier: CL rationale: "Technology, hardware, and infrastructure constraints are externally imposed." - priority: 6 tier: SAL rationale: "Architecture is bounded by all above." - priority: 7 tier: ICL rationale: "Contracts derive from architecture." - priority: 8 tier: CDL rationale: "Design derives from contracts." - priority: 9 tier: ISL rationale: "Scaffolding derives from design." override_principle: > Higher-priority tiers override lower-priority tiers. An XPD ethical boundary functions as an absolute veto right over any downstream decision. intra_tier_conflict_rule: > When two or more nodes within the same tier produce conflicting constraints, the conflict must be explicitly documented and resolved before any conflicting node may transition to status ACTIVE. The VERIFY operation must detect and report intra-tier conflicts as structural violations. physical_constraint_escalation: > Constraint precedence governs design decisions, not physical impossibilities. When a higher-priority tier produces a requirement that is physically incompatible with a lower-priority tier's declared constraint (e.g. a functional requirement that exceeds declared hardware capacity), the conflict must be escalated to the authoring authority for resolution. The precedence hierarchy does not authorize silently overriding physical or externally imposed constraints.

§7 OPERATIONS PROTOCOL

operations: core_operations: - name: INSERT description: > Create node with auto-assigned ID, parent_ids, and tier-compliant content. Supports both forward (parent→child) and reverse (child→inferred parent) direction. validation_trigger: > Full atomic ruleset; parent existence; DAG cycle detection. - name: DELETE description: "Remove node; cascade orphan detection to children." validation_trigger: > Children → DIRTY; orphaned children (zero valid parent_ids) must resolve via MODIFY (re-attach), DELETE (cascade), or SUPERSEDE (replace deleted parent); manifest updated. - name: MODIFY description: "Update content; version incremented." validation_trigger: > Re-validate ruleset; re-check citations; DIRTY propagation to all descendants. - name: SUPERSEDE description: "Mark node SUPERSEDED; create replacement with new ID." validation_trigger: > Old node retains ID; new node validated; children's parent_ids auto-updated to replacement ID then set DIRTY for content re-validation; this auto-update does not cascade DIRTY to grandchildren. - name: VERIFY description: > Traverse DAG downward; validate citation chains, edge types, ID references, orphans, and contamination. validation_trigger: > Returns CLEAN or DIRTY with itemized violations. - name: VALIDATE description: "Check single node against its tier's full atomic ruleset." validation_trigger: "Returns pass/fail with specific violated rule IDs." - name: UNBUNDLE description: > Expand Express Mode group into constituent Full Mode tiers. validation_trigger: > Content allocated to correct tiers; parent_ids auto-wired. Rejects content that cannot be unambiguously assigned.

design_decision_removed_ops: > v3.1.1 defined 11 operations. RELOCATE is removed (contradicted ID immutability). ABSTRACT and CONCRETIZE are merged into INSERT with a direction parameter. DETECT_ORPHAN and DETECT_CONTAMINATION are subsumed by VERIFY.

dirty_flag_triggers: - trigger: "Node modified" nodes_affected: "Modified node + all descendants." - trigger: "Node deleted" nodes_affected: "All former children of the deleted node." - trigger: "Parent → SUPERSEDED (auto-update of child parent_ids)" nodes_affected: > Immediate children only; grandchildren not cascaded (structural re-wiring, not semantic content change). - trigger: "CL constraint added or modified" nodes_affected: "SAL + all SAL descendants." - trigger: "XPD ethical boundary modified" nodes_affected: "All tiers (full re-validation required)."

dirty_flag_notes: - > Node Insertion: INSERT may produce nodes in one of two modes: (a) validated insertion produces an ACTIVE node synchronously or fails atomically; (b) draft insertion (via validate=false override) produces a DRAFT node. DRAFT nodes must undergo a successful VALIDATE operation to transition to ACTIVE. DRAFT nodes are structurally present in the DAG but excluded from CLEAN compliance checks. - > Supersede Auto-Update: The SUPERSEDE auto-update of child parent_ids is a structural re-wiring operation. The grandchild's inherited content remains valid pending the child's own re-validation. This scoped propagation is an explicit exception to the general MODIFY cascade rule. - > Supersede-to-MODIFY Interaction: If a DIRTY child's re-validation results in a content MODIFY, standard MODIFY cascade rules apply — all descendants of the modified child are set DIRTY. This interaction is not an exception to the SUPERSEDE scoped propagation rule; it is a consequence of the child's own MODIFY, which triggers the general cascade independently. - > Deprecation Lifecycle: A node is set to DEPRECATED via MODIFY when it is scheduled for removal or replacement. DEPRECATED nodes remain structurally valid and are included in VERIFY traversals. DEPRECATED is not a terminal state — a DEPRECATED node may subsequently be SUPERSEDED (creating a replacement) or DELETED. DEPRECATED means "scheduled for replacement, no replacement yet exists"; SUPERSEDED means "replacement exists and children have been re-wired."

resolution_workflow: > DETECT CHANGE → SET DIRTY → SCAN DOWNSTREAM → GENERATE PENDING ITEMS (node ID + violated rule ID + suggested operation) → EXECUTE OPERATION → VERIFY → SET CLEAN | REPEAT

reconciliation_manifest_tracks: - "Total node count by tier" - "ACTIVE / DIRTY / DRAFT / DEPRECATED counts" - "Pending items list" - "Last full validation timestamp" - "Active Extensions and annotation counts"

§8 EXTENSION SYSTEM

extension_system: architecture_description: > Extensions are orthogonal read-only overlays attaching to the Core DAG via extends edges. They interact with Core nodes without modifying Core semantics. permitted_actions: - "Read Core node content." - "Annotate Core nodes with namespaced metadata (stored in extension_annotations only)." - "Generate derived external artifacts (reports, IaC, recommendations)." - "Add advisories to the reconciliation manifest's extension_advisories section." prohibited_actions: - "Modify any Core node's content, parent_ids, tier, or status." - "Redefine Core tier semantics or atomic rules." - "Introduce structural cycles." - "Set Core nodes to DIRTY (advisories only; no state mutation)." candidate_pool: description: > The AI Upward Reconstruction Extension (ARE) requires special handling because it infers new nodes. To preserve AX-6, ARE-inferred nodes are placed in an Extension Candidate Pool — a staging area outside the Core DAG. candidate_status_value: "CANDIDATE (not a Core status value)" visibility_rule: "Visible only when the ARE extension is active." effect_on_core_status: "No effect on Core DIRTY/CLEAN status." promotion_mechanism: > Must be promoted into the Core DAG via INSERT (triggering full validation) to become Core nodes. discard_trigger: "Automatically discarded when ARE is disabled." integration_rules: - rule_id: EXT-R1 statement: "Must declare contract version compatible with DDR-Core-4.x." - rule_id: EXT-R2 statement: "Must declare which Core tiers it reads and which it annotates." - rule_id: EXT-R3 statement: > Annotations must be namespaced by Extension ID (e.g. 'HRE::min_hardware_profile'). - rule_id: EXT-R4 statement: > Extensions update the reconciliation manifest; annotation counts tracked. - rule_id: EXT-R5 statement: "Disabling an Extension leaves Core CLEAN/DIRTY status unchanged." - rule_id: EXT-R6 statement: > Extension-internal derived artifact graphs must maintain their own acyclicity. - rule_id: EXT-R7 statement: "Extension advisories do not mutate Core node status." normative_notes: - > 'All Core tiers' is not a valid contract declaration. Extensions must enumerate tiers by name to preserve auditability under EXT-R2.

§9 EXTENSION CATALOG

extension_catalog:

- id: E1 name: "Hardware & Resource Intelligence Extension (HRE)" contract: "HRE-1.0 / DDR-Core-4.x" reads: [CL, SAL, CDL, ISL] annotates: [CL, SAL] rules:
 - rule_id: HRE-R1 statement: > Bottom-up inference produces minimum hardware profiles as CL-compatible declarations.
 - rule_id: HRE-R2 statement: > Cloud recommendations include ≥2 provider-agnostic instance class options.
 - rule_id: HRE-R3 statement: > Top-down enforcement validates SAL patterns do not exceed CL ceilings.
 - rule_id: HRE-R4 statement: > All recommendations are advisory; they do not override CL without explicit MODIFY.

- id: E2 name: "Dependency Graph Analyzer (DGA)" contract: "DGA-1.0 / DDR-Core-4.x" reads: [CL, ICL, CDL, ISL] annotates: [CL, ICL] rules:
 - rule_id: DGA-R1 statement: > Produces a complete directed dependency graph for all CL-declared libraries.
 - rule_id: DGA-R2 statement: "Detects version conflicts with resolution suggestions."
 - rule_id: DGA-R3 statement: > Transitive dependency reports flag all copyleft licenses that could impose constraints.
- id: E3 name: "Lifecycle & Versioning Engine (LVE)" contract: "LVE-1.0 / DDR-Core-4.x" reads: [XPD, SIL, GPCL, FCL, CL, SAL, ICL, CDL, ISL] annotates: [XPD, SIL, GPCL, FCL, CL, SAL, ICL, CDL, ISL] rules:
 - rule_id: LVE-R1 statement: > Every node modification produces a version history entry with timestamp, author, and rationale.
 - rule_id: LVE-R2 statement: > Technical debt items classified by tier origin and estimated remediation effort.
 - rule_id: LVE-R3 statement: > Deprecation requires a sunset date and migration path before node → DEPRECATED.
 - rule_id: LVE-R4 statement: > Version control integration maps DDR node IDs to VCS commit hashes.
- id: E4 name: "Observability & Runtime Engine (ORE)" contract: "ORE-1.0 / DDR-Core-4.x" reads: [GPCL, SAL, ICL, CDL, ISL] annotates: [ISL, SAL] rules:
 - rule_id: ORE-R1 statement: > Telemetry stubs derived from GPCL latency and throughput targets.
 - rule_id: ORE-R2 statement: "Alert rules expressed in vendor-agnostic format."
 - rule_id: ORE-R3 statement: > Every SAL component must have ≥ 1 telemetry point for operational readiness.
 - rule_id: ORE-R4 statement: > Incident-to-design traceability maps runtime anomalies to ISL, CDL, and SAL nodes.
- id: E5 name: "AI Upward Reconstruction Engine (ARE)" contract: "ARE-1.0 / DDR-Core-4.x" reads: [ISL, CDL, ICL, SAL] annotates: [SAL, ICL, CDL, ISL] notes: > ARE annotation is restricted to tiers at or below SAL (SAL, ICL, CDL, ISL). ARE must not annotate XPD, SIL, GPCL, or FCL nodes — these tiers are above the architecture layer and must not receive AI-inferred annotations. Inferred insights pertaining to intent, governance, ethical, or functional dimensions are surfaced as Candidate Pool nodes only, subject to human promotion via INSERT. rules:
 - rule_id: ARE-R1 statement: > All inferred nodes placed in the Extension Candidate Pool (§8.2); automatic promotion prohibited.
 - rule_id: ARE-R2 statement: > Each candidate carries ARE::confidence_score (0.0–1.0) derived from source evidence quality.
 - rule_id: ARE-R3 statement: > Promotion into Core DAG requires INSERT with full atomic ruleset validation.
 - rule_id: ARE-R4 statement: > ARE must never autonomously create XPD or GPCL nodes — ethical and regulatory content requires human authorship.
- id: E6 name: "Security & Compliance Engine (SCE)" contract: "SCE-1.0 / DDR-Core-4.x" reads: [GPCL, CL, SAL, ICL] annotates: [GPCL, SAL, ICL] rules:
 - rule_id: SCE-R1 statement: > Threat models expressed in STRIDE format or equivalent structured notation.
 - rule_id: SCE-R2 statement: > Trust boundary violations in SAL flagged as high-priority advisories.
 - rule_id: SCE-R3 statement: > Every ICL contract must have an explicit RBAC access control policy.
 - rule_id: SCE-R4 statement: > PII data flows enumerated in ICL and traceable to GPCL data-residency constraints.
 - rule_id: SCE-R5 statement: "Compliance evidence records are immutable once generated."
- id: E7 name: "Data Domain Extension (DDE)" contract: "DDE-1.0 / DDR-Core-4.x" reads: [FCL, GPCL, SAL, ICL, CDL] annotates: [ICL, SAL, FCL] notes: > DDE annotates FCL to flag functional capabilities that imply data domain schemas not yet formally specified in ICL. This is a forward-reference advisory, not inference about intent. rules:
 - rule_id: DDE-R1 statement: > Canonical ER model expressed in formal notation (ERD, DBML, or equivalent).
 - rule_id: DDE-R2 statement: > Every ICL payload schema validated against the canonical ER model.
 - rule_id: DDE-R3 statement: > Schema consistency violations flagged as blocking advisories.
 - rule_id: DDE-R4 statement: > Data lifecycle policies specify retention periods traceable to GPCL regulatory requirements.
- id: E8 name: "Deployment & CI/CD Planner (DCP)" contract: "DCP-1.0 / DDR-Core-4.x" reads: [CL, SAL, ISL] annotates: [ISL, SAL] rules:

- rule_id: DCP-R1 statement: > Deployment manifests map every SAL subsystem to a deployment unit.
 - rule_id: DCP-R2 statement: > CI/CD pipeline definitions include at minimum: lint, test, build, deploy stages.
 - rule_id: DCP-R3 statement: > All generated IaC cites the CL nodes from which configuration was derived.
 - rule_id: DCP-R4 statement: > Environment-specific configuration separated from application code.
- id: E9 name: "Ethics & Human-Centered Design Extension (EHD)" contract: "EHD-1.0 / DDR-Core-4.x" reads: [XPD, SIL, FCL, SAL, CDL] annotates: [FCL, CDL, SAL] rules:
 - rule_id: EHD-R1 statement: > Bias impact assessments identify affected demographic groups and potential algorithmic biases.
 - rule_id: EHD-R2 statement: > Accessibility compliance validates FCL capabilities against WCAG 2.1 AA or GPCL-declared standard.
 - rule_id: EHD-R3 statement: > Algorithmic accountability maps link each automated CDL decision to a human oversight mechanism.
 - rule_id: EHD-R4 statement: > All EHD assessments cite the XPD ethical boundary conditions being evaluated.
 - rule_id: EHD-R5 statement: > When XPD is inactive, EHD creates a synthetic XPD-equivalent assessment anchored to SIL. The synthetic XPD-equivalent is a risk-flagging artifact only, carries no precedence weight in §6 conflict resolution, cannot be cited in Core node parent_ids, and does not substitute for a human-authored XPD node. If it identifies risks that require formal governance, it must surface a blocking advisory recommending XPD activation.

§11 COMPLIANCE CHECKLIST

compliance_checklist: structural_validation: - "All non-root nodes have ≥ 1 valid, non-superseded parent_id." - "All parent_ids reference nodes of the correct parent tier." - "No cycles exist in any citation path (VERIFY confirms)." - "No tier-skipping detected." - "All inline [TIER-N.M] citations have matching entries in parent_ids." - "No node has status DIRTY." - "Reconciliation manifest shows zero pending items." - > If any Extension is active, all Extension advisories classified as critical or blocking have a recorded disposition note. atomic_rule_validation: - "XPD nodes satisfy XPD-R1 through XPD-R6 and XPD-E1 through XPD-E3." - "SIL nodes satisfy SIL-R1 through SIL-R6 and SIL-E1 through SIL-E4." - "GPCL nodes satisfy GPCL-R1 through GPCL-R10 and GPCL-E1 through GPCL-E3." - "FCL capabilities are user-observable and free of implementation references." - "CL nodes are declarative only; no inference (CL-E1)." - "SAL cites all active parent tiers (FCL + CL if active)." - "ICL schemas are machine-parseable (ICL-R2)." - "ISL stubs contain traceable docstrings citing CDL parent IDs." - > CDL nodes produce language-specific blueprints when CL declares multiple targets (CDL-R7). extension_validation: - "All active Extensions declare compatible contract versions for DDR-Core-4.x." - "Extension annotations stored in extension_annotations only." - "Extension advisories reviewed; non-critical advisories have disposition notes." - > ARE-generated candidates reviewed and either promoted via INSERT or discarded.

GLOSSARY

- glossary:
- term: "Atomic Rule" definition: > An inclusion or exclusion constraint on a tier node, individually verifiable without reference to other nodes.
 - term: "Candidate Pool" definition: > Extension-managed staging area for ARE-inferred nodes; explicitly outside the Core DAG until promoted via INSERT.
 - term: "DAG" definition: > Directed Acyclic Graph — the DDR System's foundational data structure.
 - term: "Dirty Flag" definition: > DIRTY status indicating a node requires re-validation following a graph-modifying event.
 - term: "Edge Type" definition: > One of four typed relationships: derives, constrains, implements, extends.
 - term: "Express Mode" definition: > A four-group consumption mode; groups are unbundleable to Full Mode tiers via UNBUNDLE.
 - term: "Extension" definition: > An optional analytical overlay that reads and annotates Core nodes without modifying Core semantics.
 - term: "Leaf Node" definition: > A node with no children; ISL nodes are the only valid leaf nodes in a CLEAN Core DAG. During incremental authoring, non-ISL tiers may temporarily be leaf nodes; VERIFY flags them as incomplete.
 - term: "Merge Node" definition: > SAL — the point where FCL derivations and CL constraints converge.

- term: "Orphan" definition: > A non-root node with no valid parent_id — a structural violation.
- term: "Root Node" definition: > XPD (if active) or SIL (if XPD inactive); the only node with an empty parent_ids list.
- term: "Tier Contamination" definition: > Presence of content in a node that violates that tier's atomic exclusion rules.

— APPENDIX A: VERSION HISTORY

version_history:

- version: "1.0" date: "" summary: > Initial DDR System concept (7-tier linear: BRD→NFR→FSD→SAD→ICD→TDD→ISP).
- version: "2.1" date: "2026-02-26" summary: "Refined Core + Extension system."
- version: "3.0" date: "2026-02-26" summary: > Complete redesign: fork-join DAG; GPCL isolation; XPD optional root; Z-axis extensions; Express Mode; CRR protocol; 9 Extensions.
- version: "3.1.1" date: "2026-02-26" summary: > Structural consolidation: universal node format; 6-edge vocabulary; axiom implications.
- version: "4.0" date: "2026-02-26" summary: > Structural simplification: 11→9 tiers; 6→4 edge types; 11→7 operations; fork-join→merge-node; RELOCATE removed; ARE Candidate Pool; Express Mode→4 groups; Service Model removed; CRR protocol removed.

— APPENDIX B: TIER MIGRATION

tier_migration: policy: > All future version migrations must include a complete rule-level cross-reference table with explicit consolidation status. tier_map: - from_tier: XPD to_tier: XPD notes: "Unchanged." - from_tier: SIL to_tier: SIL notes: "Unchanged." - from_tier: GPCL to_tier: GPCL notes: "Expanded to absorb ORL quality/performance content." - from_tier: ORL to_tier: GPCL notes: > ORL-R1 through ORL-R7 become GPCL-R6 through GPCL-R10 (ORL-R5 and ORL-R6 consolidated into GPCL-R9). - from_tier: FCL to_tier: FCL notes: "Now derives from GPCL instead of ORL." - from_tier: HIL to_tier: CL notes: "HIL-R1 through HIL-R5 become CL-R6 through CL-R8." - from_tier: TDL to_tier: CL notes: "TDL-R1 through TDL-R6 become CL-R1 through CL-R5." - from_tier: SAL to_tier: SAL notes: "Simplified from fork-join to single merge-node." - from_tier: ICL to_tier: ICL notes: "Unchanged." - from_tier: CDL to_tier: CDL notes: "Unchanged." - from_tier: ISL to_tier: ISL notes: "References CL instead of TDL for language targets." rule_map: - from_rule_ids: "ORL-R1" to_rule_ids: "GPCL-R6" consolidation_status: "1:1" notes: "Maps to quantifiable performance targets rule." - from_rule_ids: "ORL-R2" to_rule_ids: "GPCL-R7" consolidation_status: "1:1" notes: "Maps to reliability and availability targets rule." - from_rule_ids: "ORL-R3" to_rule_ids: "GPCL-R8" consolidation_status: "1:1" notes: "Maps to security requirements rule." - from_rule_ids: "ORL-R4" to_rule_ids: "GPCL-R10" consolidation_status: "1:1" notes: "Maps to parent SIL citation rule." - from_rule_ids: "ORL-R5" to_rule_ids: "GPCL-R9" consolidation_status: "N:1" notes: "Consolidated with ORL-R6 into scalability and accessibility rule." - from_rule_ids: "ORL-R6" to_rule_ids: "GPCL-R9" consolidation_status: "N:1" notes: "Consolidated with ORL-R5 into scalability and accessibility rule." - from_rule_ids: "ORL-R7" to_rule_ids: "GPCL-R10" consolidation_status: "1:1" notes: "NOTE — Audit C-3: mapping marked TBD in source doc; assigned here pending board confirmation." - from_rule_ids: "HIL-R1, HIL-R2, HIL-R3" to_rule_ids: "CL-R6" consolidation_status: "N:1 Consolidated" notes: "Consolidated into hardware envelopes rule." - from_rule_ids: "HIL-R4" to_rule_ids: "CL-R7" consolidation_status: "1:1" notes: "" - from_rule_ids: "HIL-R5" to_rule_ids: "CL-R8" consolidation_status: "1:1" notes: "" - from_rule_ids: "TDL-R1" to_rule_ids: "CL-R1" consolidation_status: "1:1" notes: "" - from_rule_ids: "TDL-R2, TDL-R6" to_rule_ids: "CL-R2" consolidation_status: "N:1 Consolidated" notes: "Consolidated into minimum version bounds rule." - from_rule_ids: "TDL-R3" to_rule_ids: "CL-R3" consolidation_status: "1:1" notes: "" - from_rule_ids: "TDL-R4" to_rule_ids: "CL-R4" consolidation_status: "1:1" notes: "" - from_rule_ids: "TDL-R5" to_rule_ids: "CL-R5" consolidation_status: "1:1" notes: ""

— CANONICAL DAG NODES (9-Tier Topology)

These nodes represent the DDR System's own tier structure as the exemplar DAG.

Each node encodes its tier's core question as content and its canonical position in the topology via parent_ids.

nodes:

- id: XPD-0.1 tier: XPD title: "Existential Purpose Document" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids: [] content: | What human or societal need does this system exist to address, and what ethical boundaries govern all downstream decisions?

The DDR System addresses the universal need for deterministic, traceable software requirements documentation — ensuring that every implementation decision can be traced to originating human intent. Ethical boundaries: the system must not be used to obscure design decisions, create post-hoc documentation that misrepresents actual system behavior, or suppress evidence of design failures. All downstream tiers must remain honest, complete, and auditable representations of system intent and implementation reality.

Populations at risk: software practitioners whose systems were built without traceable requirements; end-users of systems whose failure modes were never documented. Safeguards: the structural invariants of the DAG make omission mechanically detectable.

- id: SIL-1.1 tier: SIL title: "Strategic Intent Layer" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids:
 - id: XPD-0.1 edge_type: derives content: | Why does this system exist, and what business outcomes must it achieve?

The DDR System exists to eliminate the gap between what a software system is intended to do and what it is documented to do. Strategic objectives: (1) provide a deterministic, layered requirements framework adoptable from solo developer to enterprise scale without structural changes; (2) enable complete audit trails from business intent to implementation stub; (3) make structural violations mechanically detectable, not dependent on reviewer expertise. Scope: all software systems regardless of domain, technology, or scale (AX-4). Out of scope: commercial pricing models, deployment infrastructure, domain-specific extensions (→ Extensions).

- id: GPCL-2.1 tier: GPCL title: "Governance, Policy & Quality Layer" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids:
 - id: SIL-1.1 edge_type: derives content: | What non-negotiable external mandates, regulatory obligations, policy constraints, and measurable quality thresholds govern this system?

Governing axioms: AX-1 through AX-7 are non-negotiable structural laws. Quality thresholds: every non-root node must have a traceable parent (AX-1); graph must be acyclic at all path lengths (AX-7); identical inputs must produce unambiguous outputs (AX-3). Constraint precedence is inviolable (§6). Operations must be atomic: partial execution that leaves the DAG in an inconsistent state is prohibited (§7). Extension annotations must never modify Core content (AX-6, §8).

- id: FCL-3.1 tier: FCL title: "Functional Capability Layer" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids:
 - id: GPCL-2.1 edge_type: derives content: | What externally observable behaviors and user-facing capabilities must the system provide?

A practitioner using the DDR System must be able to: author nodes at any of the 9 defined tiers with content governed by that tier's atomic ruleset; express parent-child relationships using one of 4 typed edge types; execute VERIFY to traverse the full DAG and receive an itemized violation list; execute VALIDATE on a single node and receive pass/fail with specific rule IDs; author in Express Mode using 4 groups and UNBUNDLE any group to its constituent tiers without information loss; activate optional Extensions that annotate nodes without modifying Core content; receive DIRTY propagation automatically when any ancestor node is modified.

- id: CL-4.1 tier: CL title: "Constraint Layer" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids:

- id: FCL-3.1 edge_type: derives content: | What are the declared technology selections, hardware envelopes, and infrastructure ceilings that bound the system's implementation?

Approved data format: YAML (primary), JSON (interchange). Validation contract: JSON Schema Draft 2020-12. Graph library target: networkx 3.x (Python), igraph (R). Rendering engine: Graphviz DOT (system binary required). Prohibited: any implementation that allows Extension writes to Core node content, parent_ids, tier, or status fields. Deployment topology: stateless execution; no persistent server process required for Core operations. Runtime: Python 3.10+ or R 4.0+.

- id: SAL-5.1 tier: SAL title: "System Architecture Layer" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids:
 - id: FCL-3.1 edge_type: derives
 - id: CL-4.1 edge_type: constrains content: | How is the system structurally decomposed, and what patterns govern component interaction?

Architectural pattern: layered DAG with typed edges and merge-node at SAL. Subsystem decomposition: (1) Core DAG Engine — node schema management, ID assignment, status lifecycle enforcement, parent_ids maintenance; (2) Validation Engine — VERIFY (full traversal) and VALIDATE (per-node) with deterministic rule evaluation; (3) Operation Protocol — 7 atomic operations with pre/post-conditions and DIRTY propagation side-effects; (4) Extension Interface — read-only overlay contract, candidate pool isolation, annotation namespace enforcement; (5) Reconciliation Manifest — status counters, pending item queue, Extension advisory aggregation. Concurrency: last-write-wins. Failure isolation: each operation is fully atomic; rollback on validation failure. Resilience: VERIFY is always executable regardless of DIRTY count.

- id: ICL-6.1 tier: ICL title: "Interface & Contracts Layer" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids:
 - id: SAL-5.1 edge_type: derives content: | What are the formal, machine-verifiable contracts governing data exchange between system boundaries?

Node schema contract: ddr_node_schema.yaml (JSON Schema 2020-12); fields id, tier, title, status, version, created, modified required; parent_ids and content optional with defined defaults. Edge schema: {id: string (pattern TIER-N.M), edge_type: enum[derives|constrains|implements|extends]}. VERIFY contract: input DiGraph → output {clean: bool, violations: [{rule_id, node_id, description, severity}]}. VALIDATE contract: input node_id → output {pass: bool, violated_rules: [rule_id]}. INSERT contract: input node_spec → output DdrNode | ValidationError (atomic). Error codes: CIT-R1 (orphan), AX-7 (cycle), INV-TIER-SKIP (tier skip), SCHEMA-STATUS (invalid status). Versioning: schema locked to ddr_version: "4.0"; breaking schema changes require ddr_version increment.

- id: CDL-7.1 tier: CDL title: "Component Design Layer" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids:
 - id: ICL-6.1 edge_type: implements content: | What are the structural blueprints of individual components?

DdrNode: id (str, immutable), tier (TierEnum), title (str), status (StatusEnum: DRAFT|ACTIVE|DIRTY|DEPRECATED|SUPERSEDED), version (SemVer), created (datetime), modified (datetime), parent_ids (List[ParentCitation]), content (str), extension_annotations (Dict[str, Any], namespace). ParentCitation: id (str), edge_type (EdgeTypeEnum). Violation: rule_id (str), node_id (str), description (str), severity (str). VerifyResult: clean (bool), violations (List[Violation]), node_count (int), tier_counts (Dict), status_counts (Dict). ReconciliationManifest: node_count_by_tier (Dict), status_counts (Dict), pending_items (List), last_validation_ts (datetime), active_extensions (List), annotation_counts (Dict), extension_advisories (List). Dependencies: DdrNode → ParentCitation (composition); VerifyResult → Violation (composition); ReconciliationManifest → pending_items (list of {node_id, rule_id, suggested_op}).

- id: ISL-8.1 tier: ISL title: "Implementation Scaffold Layer" status: ACTIVE version: "4.0.0" created: "2026-02-26T00:00:00Z" modified: "2026-02-26T00:00:00Z" parent_ids:
 - id: CDL-7.1 edge_type: implements content: |

[CDL-7.1] DdrNode scaffold — Python target (CL-4.1: Python 3.10+)

from enum import Enum from datetime import datetime from dataclasses import dataclass, field from typing import Any

class TierEnum(str, Enum): # [GPCL-2.1] 9 canonical tiers — AX-4 universality enforced XPD="XPD"; SIL="SIL"; GPCL="GPCL"; FCL="FCL"; CL="CL" SAL="SAL"; ICL="ICL"; CDL="CDL"; ISL="ISL"

```
class StatusEnum(str, Enum): # [ICL-6.1] Valid statuses — transitions enforced by operation protocol DRAFT="DRAFT"; ACTIVE="ACTIVE"; DIRTY="DIRTY" DEPRECATED="DEPRECATED"; SUPERSEDED="SUPERSEDED"

class EdgeTypeEnum(str, Enum): # [ICL-6.1] 4-type edge vocabulary (§3.2) DERIVES="derives"; CONSTRAINS="constrains" IMPLEMENTS="implements"; EXTENDS="extends"

@dataclass class ParentCitation: # [ICL-6.1] Typed parent reference — CIT-R1 through CIT-R4 id: str # immutable node ID — pattern TIER-N.M edge_type: EdgeTypeEnum

@dataclass class DdrNode: # [ICL-6.1] Core node — AX-1: parent_ids ≥ 1 for non-root id: str # [§3.6] immutable on assignment tier: TierEnum title: str status: StatusEnum version: str # SemVer created: datetime modified: datetime parent_ids: list[ParentCitation] = field(default_factory=list) content: str = "" extension_annotations: dict[str, Any] = field(default_factory=dict)

@dataclass class Violation: # [ICL-6.1] VERIFY output unit rule_id: str; node_id: str; description: str; severity: str = "ERROR"

@dataclass class VerifyResult: # [ICL-6.1] VERIFY return contract clean: bool violations: list[Violation] = field(default_factory=list) node_count: int = 0 tier_counts: dict = field(default_factory=dict) status_counts: dict = field(default_factory=dict)

def verify(graph) -> VerifyResult: "[SAL-5.1] Full DAG traversal — AX-7 cycle, AX-1 orphan, INV-2 tier-skip, edge type, SAL merge-node, status validation." ... # implementation hint: use networkx.is_directed_acyclic_graph()

def validate(node_id: str, graph) -> dict: "[SAL-5.1] Per-node atomic ruleset check — returns {pass, violated_rules}." ... # implementation hint: load tier_definitions ruleset from schema

def insert(node_spec: dict, graph, validate: bool = True) -> DdrNode: "[SAL-5.1] Atomic node creation — fails atomically on validation failure." ...

def delete(node_id: str, graph) -> None: "[SAL-5.1] Remove node; propagate DIRTY to all former children." ...

def modify(node_id: str, changes: dict, graph) -> DdrNode: "[SAL-5.1] Update content; increment SemVer; propagate DIRTY to descendants." ...

def supersede(node_id: str, replacement_spec: dict, graph) -> DdrNode: "[SAL-5.1] Mark SUPERSEDED; create replacement; re-wire children parent_ids." ...

def unbundle(group_id: str, graph) -> list[DdrNode]: "[SAL-5.1] Expand Express Mode group; reject ambiguous content; auto-wire parent_ids." ...
```